

Experiencia VJ-1

[0 - Prerrequisitos y el Manual](#)

[1 - Abrir el Proyecto en VS Code](#)

[2 - El Proyecto y Código Base 🍷 \(INFO\)](#)

[2.1 - Y que es todo esto??? 😞 \(INFO\)](#)

[2.2 - Pantalla y Manejo de Escenas \(INFO\)](#)

[2.2 - El Gameplay \(TODO\)](#)

[2.3 - basic_scene.py \(INFO\)](#)

[2.4 - game_scene.py \(INFO\)](#)

[2.5 - El Display \(TODO\)](#)

[2.6 - Jorge y los BUGS \(INFO\)](#)

[2.6 - Jorge y los BUGS \(TODO\)](#)

[2.7 - Jorge y los BUGS Parte 2 \(TODO\)](#)

[2.8 - Generando BUGS \(TODO\)](#)

[2.9 - El Tiempo \(TODO\)](#)

[2.10 - Esquivando BUGS \(TODO\)](#)

[3 - JUGAR](#)

[3.1 - Para el Entregable \(TODO\)](#)

0 - Prerrequisitos y el Manual

Para el desarrollo de esta experiencia necesitan tener instalada la librería **pygame-ce** . Pueden revisar la instalación ejecutando:

```
python -m pygame.example.aliens
```

Si no les reconoce el comando **python** pueden probar con alguna de **py py3 python3**

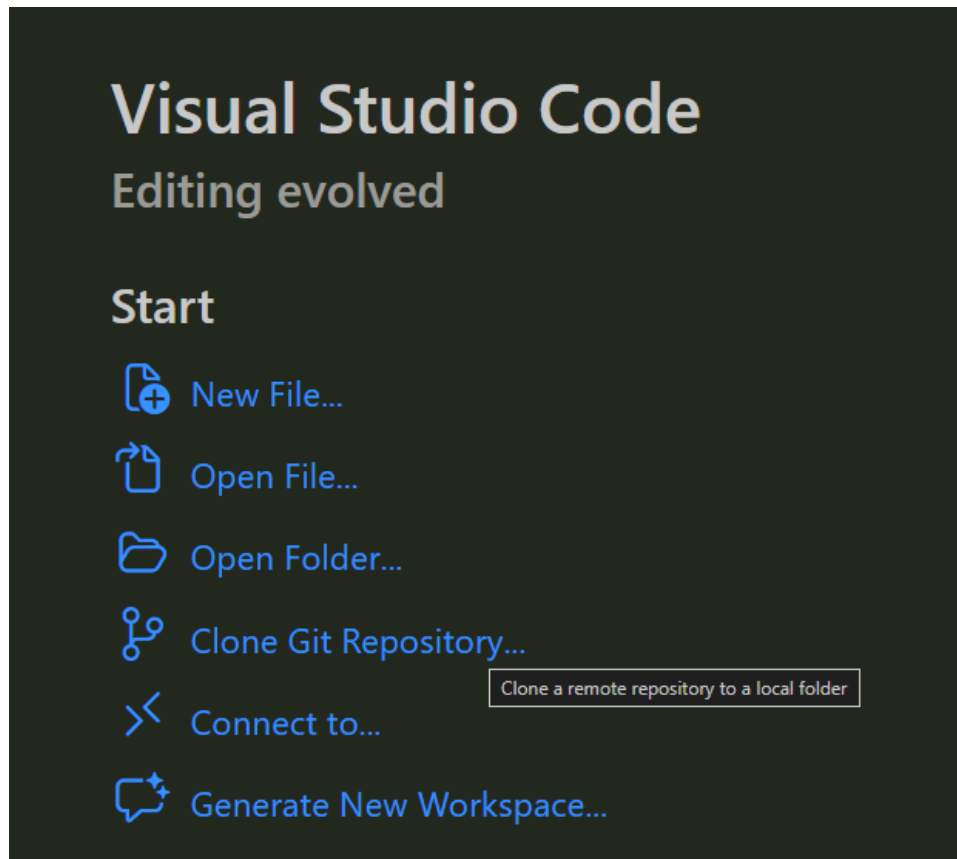
En este manual encontraras dos tipos de secciones:

- **INFO:** Contiene información y explica el funcionamiento del código base.
- **TODO:** Contiene las diversas tareas por hacer (*to-do*) para el entregable.

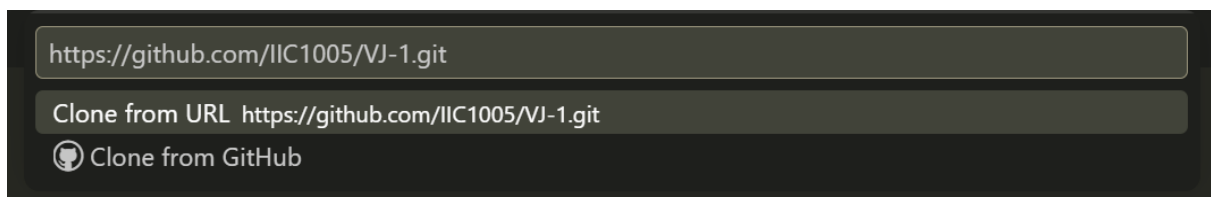
Todos los **TODO** tienen un comentario asociado en el código, indicando el lugar donde tendrán que completar.

1 - Abrir el Proyecto en VS Code

Tal como vimos en la experiencia GIT, deben clonar el [repositorio base](#) desde GitHub.

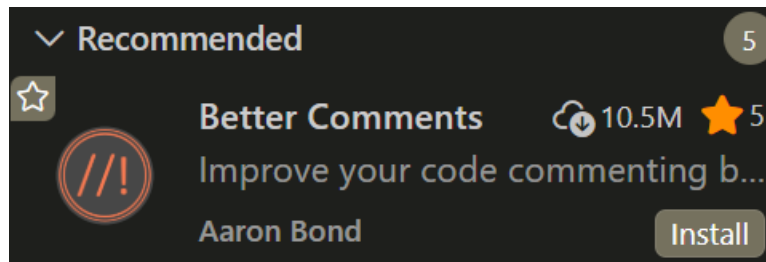


En una nueva ventana de VS Code, le damos a "Clone Git Repository..."



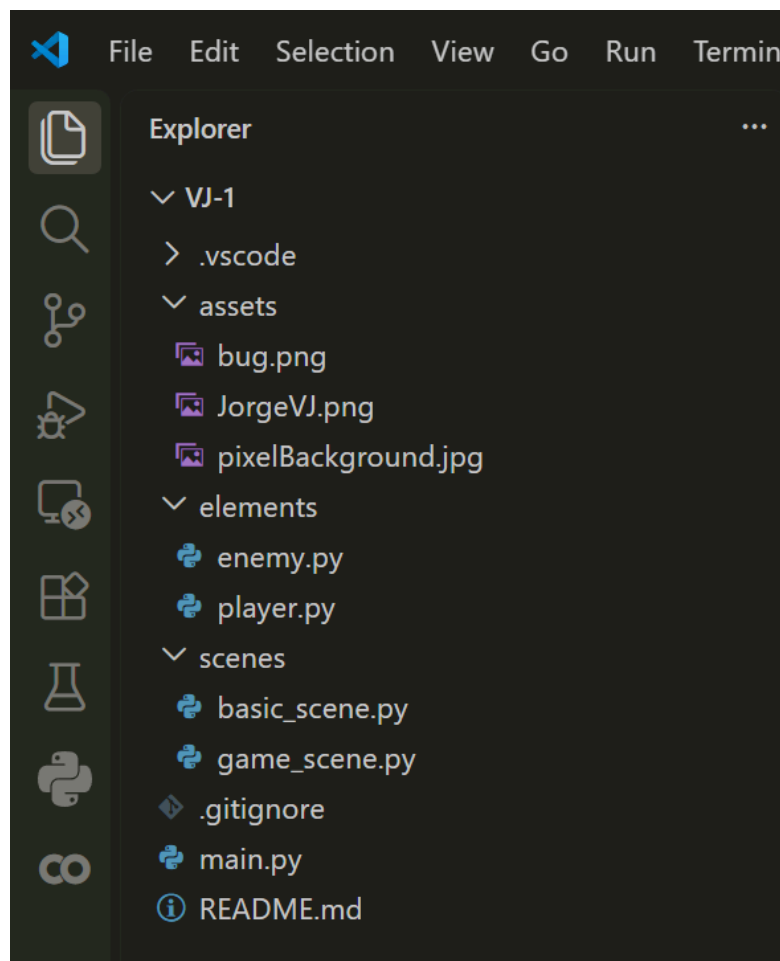
Pegamos el link y clonamos. El explorador les pedirá donde guardar el repositorio.

RECOMENDADO: Si van al buscador de extensiones en la barra lateral de VS Code, les debería aparecer "Better Comments" en la sección *Recommended*. Esta extension permite colorear los comentarios en el código y así coordinarlo con el código de color del manual.



2 - El Proyecto y Código Base 🧩 (INFO)

Una vez clonado y abierto el repositorio, deberían ver la siguiente estructura en VS Code.



Estructura base del proyecto.

Todos estos archivos separados se pueden ver intimidantes, pero no se preocupen! Ordenar el código en módulos es solo una buena practica y nos permitirá visualizar y organizar los componentes del juego de mejor manera.

2.1 - Y que es todo esto??? 😓 (INFO)

El repositorio contiene **todos los archivos y una buena parte del código** necesario para que puedan programar su juego. El proyecto se encuentra organizado de una manera particular, con 3 **módulos** (carpetas) principales y el archivo **main.py**.

- **assets/** : Contiene todas las **imágenes** (fondos, jugador, enemigos) que vamos a usar en el proyecto.
- **elements/** : Contiene los archivos que modelan la **lógica de los elementos** del juego (jugador y enemigos). Vamos a usar un archivo por elemento.
- **scenes/** : Contiene los archivos donde se programa la **lógica de las escenas** del juego. Todo evento ocurre dentro de una escena y cada escena tiene su propio archivo.
- **main.py** : El punto de entrada a nuestro juego. Es aquí donde **manejamos las diferentes escenas**. Este es el archivo que se debe ejecutar para correr el juego.

2.2 - Pantalla y Manejo de Escenas (INFO)

Para la experiencia **PA**, programaron algunos juegos, pero estos se jugaban a través del terminal. Para esta experiencia vamos a ir mas allá y usaremos interfaces graficas. Puede que suene complicado (y lo es), así que usaremos una librería que se encargue de todo esto: **pygame**.

```

main.py
main.py > ...
1  import pygame
2
3  from scenes import basic_scene, game_scene
4
5  # ? Inicializamos pygame
6  pygame.init()
7
8  # ? Definimos las medidas de nuestra pantalla
9  SCREEN_WIDTH = 1000
10 SCREEN_HEIGHT = 700
11
12 # ? Creamos nuestro objeto pantalla
13 screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
14
15 # ? Aquí se ejecutarán las escenas del juego en orden
16 basic_scene.gameloop(screen)
17
18 # TODO (2.2): Añadir nueva escena de juego
19

```

Código base en main.py

pygame nos permite fácilmente definir un objeto **screen**, creando una ventana para que nuestro juego exista. Importamos la librería, definimos las dimensiones de nuestra ventana deseada y la creamos llamando **pygame.display.set_mode**. Ahora podemos entregarle este objeto a nuestras escenas para que tengan una ventana sobre la cual puedan dibujar.

Si ejecutan el programa, verán simplemente una pantalla en negro con algunas instrucciones básicas. Esta es la escena **basic_scene**.

2.2 - El Gameplay (TODO)

Y que juego hacemos solo con la pantalla en negro?

Ninguno 😊

Es por eso que añadiremos una nueva escena a nuestro juego!

En **main.py** añadiremos una nueva línea, que usará la escena **game_scene**, la cual, por ahora, está casi vacía.

```
# ? Aqui se ejecutaran las escenas del juego en orden
basic_scene.gameloop(screen)

# TODO (2.2): Añadir nueva escena de juego
game_scene.gameloop(screen)
```

Agregamos a main.py

Cada vez que cerramos una ventana, pasaremos a la siguiente escena. Es decir, si queremos pasar a la **game_scene** apretamos **ESC** una vez para cerrar la **basic_scene**. Si queremos salir de todas las escenas apretamos **ESC** dos veces.

2.3 - basic_scene.py (INFO)

Es una escena base que no sera necesario que editen. Si revisan el código, notaran que solo define una función **gameloop(screen)**. Todas las escenas deben definir esta función, la cual corresponderá al *loop* de eventos que ocurren dentro de esta.

En este caso, simplemente espera que se cierre la ventana para poder pasar a la siguiente escena: **game_scene.py**.

2.4 - game_scene.py (INFO)

Esta sera la escena que le da forma a nuestro juego. Es aquí donde tendrán que programar la mayor parte de la lógica del juego. (no se preocupen, sera 99% guiado)

Vamos a tener a nuestro personaje (Jorge) moviéndose por toda la ventana, esquivando obstáculos (Bugs) que aparecerán de manera aleatoria a los costados de nuestra ventana.

Si un Bug le alcanza a dar a Jorge el juego termina.

Y como traducimos esta lógica a código? Vamos a tener dos clases: **Player** y **Enemy**

- **Player** va a poder moverse mientras este dentro de la ventana.
- **Enemy** aparecerá cada cierto tiempo en el borde de nuestra ventana.
- Si **Player** impacta **Enemy**, se cerrará la venta

En primer lugar, importamos los **elementos** necesarios para nuestra escena (los cuales veremos mas adelante) y algunas teclas y eventos desde **pygame.locals**

```
import pygame
from pygame.locals import K_ESCAPE, KEYDOWN, QUIT

from elements.enemy import Enemy
from elements.player import Player
```

scenes/game_scene.py

- **KEYDOWN** representa el evento de presionar cualquier tecla
- **QUIT** representa el evento donde se cierra la ventana
- **K_ESCAPE** representa la tecla ESC

Existen muchas teclas mas, como K_1, K_UP, K_W, K_LSHIFT, etc.

Luego, al igual que **basic_scene** , define la función **gameloop(screen)**, la cual se divide en dos secciones:

Definición de Variables

Definimos algunas variables y objetos necesarios para la escena. Actualmente tienen asignado el valor **None**, mas adelante tendrán que modificar esto.

```
def gameloop(screen):
    # * Preparamos la escena de juego, cargando los elementos que se van a usar en el loop principal

    # TODO (2.5): Añadir fondo del display
    background_image = None

    # TODO (2.7): Crear la instancia de jugador
    player = None

    # TODO (2.7): Crear los grupos de sprites
    enemies = None
    all_sprites = None

    # TODO (2.8): Crear el generador de enemigos
    ADDENEMY = None

    # TODO (2.9): Crear el reloj del juego
    clock = None
```

Primera parte del *gameloop* en scenes/game_scene.py

Estas serán todas las variables que usaran en el *gameloop*, no es necesario agregar mas.

Loop Principal

Se abre el loop principal del juego. Dentro de este debemos manejar todos los eventos que ocurran en nuestro juego.

```
# * Loop principal del juego, todo lo que ocurre en el juego se hace dentro de este loop
while running:
    # TODO (2.5): Dibujar la imagen de fondo en la ventana

    # Iteramos sobre cada evento en la cola
    for event in pygame.event.get():
        if event.type == KEYDOWN: # se presiono una tecla?
            if event.key == K_ESCAPE: # era la tecla de escape?
                running = False # terminamos el loop

            elif event.type == QUIT: # fue un click al cierre de la ventana?
                running = False # terminamos el loop

        # TODO (2.8): Generar enemigos

    # TODO (2.7): Actualizar el estado interno de los sprites (posiciones, etc)

    # TODO (2.7): Dibujar los sprites actualizados en la ventana

    # TODO (2.10): Calcular colisiones entre jugador y enemigos

    # TODO (2.5): Actualizar la ventana para reflejar todos los cambios

    # TODO (2.9): Controlar la velocidad de fotogramas
```

Segunda parte del *gameloop* en *scenes/game_scene.py*

Todos los eventos que ocurran en nuestro loop serán capturados por la función **pygame.event.get()**

Por ejemplo, ya les entregamos la funcionalidad para cerrar la ventana.

- Iteramos los eventos en **pygame.event.get()**
- Revisamos si se cerró la ventana o se presiono la tecla **ESC**
- Si se cumple alguna de las anteriores, terminamos el loop

Es importante que todo evento ocurra dentro del **while running**. Si queremos detener el loop, basta con cambiar el valor de esta variable a **False**.

2.5 - El Display (TODO)

Actualmente, nuestra escena no tiene absolutamente nada. Tenemos que darle un lugar a Jorge para que pueda moverse. Y cual es un buen lugar para que

Jorge esquive los BUGS? Pues dentro de un computador!

Para el fondo del juego van a usar la imagen que les dimos en la carpeta

assets: `pixelBackground.jpg`

Nos vamos a `game_scene.py` y cargamos la imagen con `pygame.image.load`

```
# TODO (2.5): Añadir fondo del display
background_image = pygame.image.load("assets/pixelBackground.jpg").convert()
```

scenes/game_scene.py

Que hace `pygame.image.load()`? Recibe la ruta de la imagen que queremos cargar y la guarda en la variable asignada.

Que hace `.convert()`? Escalara la imagen para que sea proporcional con el tamaño de la ventana. En este caso no hará nada ya que la ventana que creamos es del mismo tamaño que la imagen.

Pero esto no es todo, ahora debemos **dibujar** la imagen en la ventana, así que en dentro del *game-loop* agregamos

```
# * Loop principal del juego, todo lo que ocurre en el juego se hace dentro de este loop
while running:
    # TODO (2.5): Dibujar la imagen de fondo en la ventana
    screen.blit(background_image, (0, 0))
```

scenes/game_scene.py

Que hace `.blit()`? Toma dos argumentos, el QUE se dibujara y el DONDE se dibujara en la superficie que escojamos, en este caso, screen.

IMPORTANTE: La esquina superior izquierda corresponde a la coordenada (0, 0). Incrementar en el eje X nos mueve hacia la derecha, mientras que incrementar en el eje Y nos mueve hacia abajo.

Ya tenemos lo que vamos a dibujar y donde. Lo único que nos falta es actualizar la pantalla. Para esto utilizamos

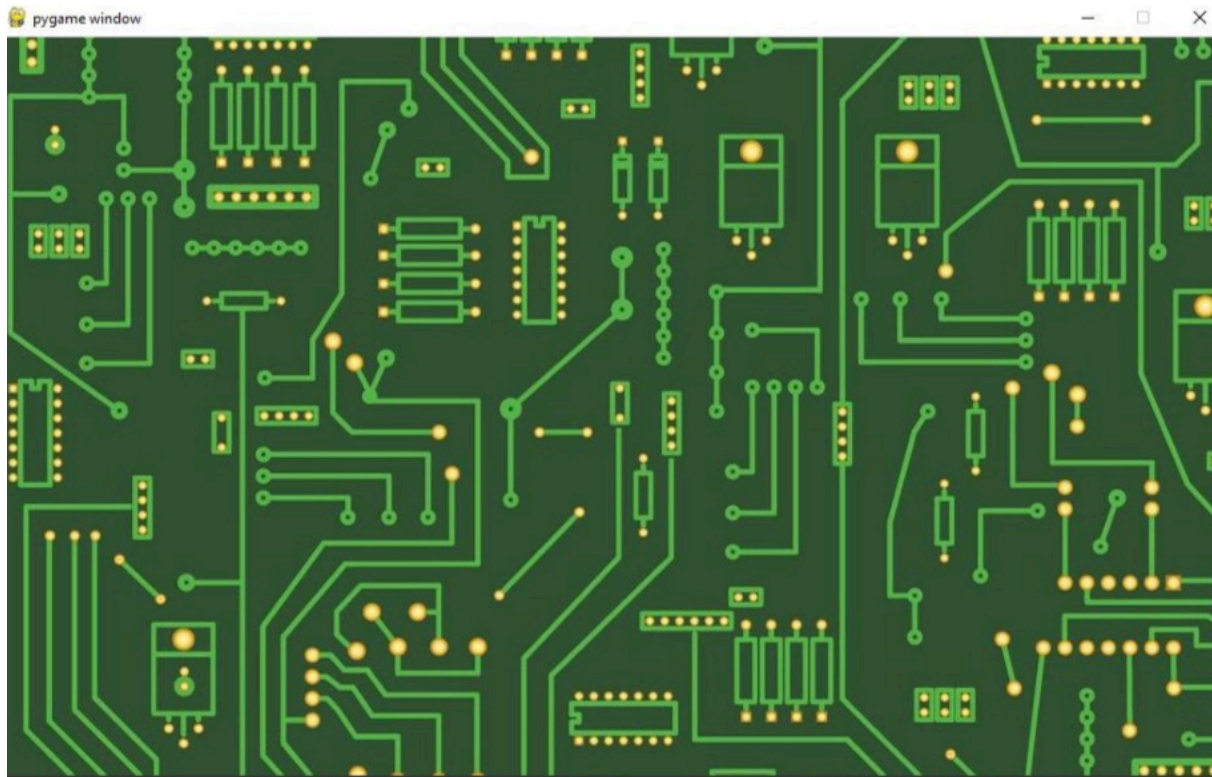
```
# TODO (2.5): Actualizar la ventana para reflejar todos los cambios
pygame.display.flip()
```

scenes/game_scene.py

Que hace **pygame.display.flip()**? Actualiza la ventana con todo lo que le hemos dibujado usando **.blit()**.

Y listo! Con esto ya tenemos nuestra ventana para el juego.

Ahora, cuando ejecutemos **main.py** y salgamos de **basic_scene**, debería aparecer la siguiente ventana con nuestra imagen de fondo.



Ventana actual de la game_scene

2.6 - Jorge y los BUGS (INFO)

Ya tenemos un fondo, pero, tenemos un juego?

El siguiente paso es agregarle interacción, añadiendo a Jorge y los BUGS.

Para esto, ya les entregamos 2 archivos en el modulo [elements/](#)

```

elements > player.py > ...
1  import pygame
2  from pygame.locals import K_DOWN, K_LEFT, K_RIGHT, K_UP
3
4  JorgePNG = pygame.image.load("assets/JorgeVJ.png")
5  JorgePNG_scaled = pygame.transform.scale(JorgePNG, (80, 80))
6
7
8  class Player(pygame.sprite.Sprite):
9      def __init__(self, screen):
10
11         # ? super().__init__() inicializa la clase padre (Sprite)
12         super().__init__()
13
14         self.image = JorgePNG_scaled
15         self.rect = self.image.get_rect()
16         self.screen_width = screen.get_width()
17         self.screen_height = screen.get_height()
18
19     def update(self, pressed_keys):
20         # TODO (2.6): Mover a Jorge
21
22         # TODO (2.6): Mantener a Jorge en Pantalla
23         pass
24

```

elements/player.py

```

elements > enemy.py > ...
1  import random
2
3  import pygame
4
5  BUGpng = pygame.image.load("assets/bug.png")
6  BUGpng_scaled = pygame.transform.scale(BUGpng, (64, 64))
7
8
9  class Enemy(pygame.sprite.Sprite):
10     def __init__(self, screen):
11
12         # ? super().__init__() inicializa la clase padre (Sprite)
13         super().__init__()
14
15         self.image = BUGpng_scaled
16         self.rect = self.image.get_rect(
17             center=(
18                 screen.get_width() + 100,
19                 random.randint(0, screen.get_height()),
20             )
21         )
22         self.speed = random.randint(3, 5)
23
24     def update(self):
25         # TODO (2.6): Mover a los enemigos
26
27         # TODO (2.6): Destruir a los enemigos
28         pass
29

```

elements/enemy.py

Ambos archivos son muy similares y, en general, todo sprite que quieran implementar seguirá esta misma estructura.

En primer lugar, se importa **pygame** y, en el caso de Jorge, las teclas que vamos a utilizar en este sprite.

```

import pygame
from pygame.locals import K_DOWN, K_LEFT, K_RIGHT, K_UP

```

Import utilizado en player.py

Luego, cargamos los *assets* que vamos a usar, en este caso, la imagen de Jorge o del BUG.

```

JorgePNG = pygame.image.load("assets/JorgeVJ.png")
JorgePNG_scaled = pygame.transform.scale(JorgePNG, (80, 80))

```

Código para cargar la imagen del sprite en elements/player.py

```

BUGpng = pygame.image.load("assets/bug.png")
BUGpng_scaled = pygame.transform.scale(BUGpng, (64, 64))

```

Código para cargar la imagen del sprite en elements/enemy.py

Que hace **pygame.image.load()**? Recibe la ruta de la imagen que queremos cargar y la guarda en la variable asignada.

Que hace **pygame.transform.scale()**? Recibe la imagen que queremos ajustar y el tamaño deseado, entregándonos la imagen escalada.

Por ultimo, ambos definen una clase **Player** y **Enemy** respectivamente. Como todo sprite que queramos crear en **pygame**, tienen 3 características principales:

Herencia de **pygame.sprite.Sprite**

```
class Player(pygame.sprite.Sprite):
```

Definición de la clase Player en elements/player.py

```
class Enemy(pygame.sprite.Sprite):
```

Definición de la clase Enemy en elements/enemy.py

“Ambas clases tienen algo raro no? Pues es que heredan, ¿Que es heredar? Aun no lo necesitais”

- Jorge (posiblemente)

Todos los sprites que queramos implementar deben heredar de la clase **Sprite** de **pygame**. Esto nos permite llamar métodos de esta clase “padre”, ahorrándonos una gran cantidad de trabajo.

Método **__init__(self, screen)**

Método para inicializar todos los parámetros y variables que vamos a usar en nuestro sprite.

```
def __init__(self, screen):

    # ? super().__init__() inicializa la clase padre (Sprite)
    super().__init__()

    self.image = JorgePNG_scaled
    self.rect = self.image.get_rect()
    self.screen_width = screen.get_width()
    self.screen_height = screen.get_height()
```

Método `__init__()` de Player en `elements/player.py`

```
def __init__(self, screen):

    # ? super().__init__() inicializa la clase padre (Sprite)
    super().__init__()

    self.image = BUGpng_scaled
    self.rect = self.image.get_rect(
        center=(
            screen.get_width() + 100,
            random.randint(0, screen.get_height()),
        )
    )
    # La velocidad también es aleatoria
    self.speed = random.randint(3, 5)
```

Método `__init__()` de Enemy en `elements/enemy.py`

Nota: `super().__init__()` inicializa la clase padre `pygame.sprite.Sprite` para que podamos usarla y aprovechar sus métodos.

Todo sprite debe definir los siguientes dos atributos:

- **self.image** : La imagen de nuestro sprite (que ya cargamos anteriormente!)
- **self.rect** : El rectángulo asociado a la imagen (esto es lo que movemos e interactúa con los demás elementos)

Obtenemos **self.rect** a partir de la imagen usando **self.image.get_rect()**. En el caso de **Enemy**, le asignamos una posición inicial aleatoria a la derecha de la pantalla para que esté fuera de vista.

Por ultimo, agregamos algunos atributos opcionales que nos van a ser útiles mas adelante. En el caso de **Player**, guardamos las dimensiones de la pantalla que le entregamos. En el caso de **Enemy**, guardamos la velocidad a la que se moverá, la cual generamos de forma aleatoria para cada instancia.

Método `update(self, *)`

Método que contendrá toda la lógica de nuestro sprite. Nuestras escenas llaman constantemente este método dentro de su *game-loop*, los cuales se encuentran actualmente vacíos.

Le podemos definir los parámetros que queramos. Por ejemplo, a **Player** le entregamos la lista con las teclas actualmente presionadas para poder moverlo, cosa que veremos a continuación.

2.6 - Jorge y los BUGS (TODO)

Para que nuestros sprites se muevan e interactúen con el juego, debemos actualizar sus métodos `update()`. Comencemos con **Player**.

En primer lugar, queremos que **Jorge** pueda moverse para poder esquivar los futuros BUGS, por lo que tenemos que pasarle el argumento `pressed_keys`, una lista con el estado actual del teclado y todas la teclas presionadas. Entonces, en `update()` agregamos

```
def update(self, pressed_keys):
    # TODO (2.6): Mover a Jorge
    if pressed_keys[K_UP]:
        self.rect.move_ip(0, -4)
    if pressed_keys[K_DOWN]:
        self.rect.move_ip(0, 4)
    if pressed_keys[K_LEFT]:
        self.rect.move_ip(-4, 0)
    if pressed_keys[K_RIGHT]:
        self.rect.move_ip(4, 0)
```

Primera parte de `update()` de **Player** en `elements/player.py`

Que hace **rect.move_ip()**? Recibe dos valores, pixeles a mover en el eje X y pixeles a mover en el eje Y. Luego, moverá el **rect** (y su image asociada) en esa cantidad. Recordar que la esquina superior izquierda corresponde al **(0, 0)**. Movimiento negativo en Y es hacia arriba.

Tampoco queremos que **Jorge** se nos escape de la pantalla, por lo que necesitamos limitar su rango de acción. Agregamos

```
# TODO (2.6): Mantener a Jorge en Pantalla
self.rect.left = max(self.rect.left, 0)
self.rect.right = min(self.rect.right, self.screen_width)
self.rect.top = max(self.rect.top, 0)
self.rect.bottom = min(self.rect.bottom, self.screen_height)
```

Segunda parte de `update()` de `Player` en `elements/player.py`. Puede hacerse con `ifs`.

self.rect.[dirección] representa el borde del **rect** en la dirección dada. Por ejemplo, **self.rect.left** seria el borde izquierdo del **rect** del sprite. Luego, nuestro borde izquierdo siempre sera mayor que 0 y se encontrara dentro de las dimensiones de nuestra ventana.

Con esto tenemos listo a nuestro **Player**.

Los **BUGS** son un poco mas simples. Estos no hay que controlarlos, solo queremos que se muevan de derecha a izquierda a lo largo de la pantalla.

```
def update(self):
    # TODO (2.6): Mover a los enemigos
    self.rect.move_ip(-self.speed, 0)
```

Primera parte de `update()` de `Enemy` en `elements/enemy.py`

Que hace **rect.move_ip()**? Recibe dos valores, pixeles a mover en el eje X y pixeles a mover en el eje Y. Luego, moverá el **rect** (y su image asociada) en esa cantidad. En este caso solo lo movemos horizontalmente según la velocidad definida de forma aleatoria anteriormente.

Por ultimo, necesitamos destruir los **BUGS** que se encuentren fuera de la pantalla. Para esto, vemos si su extremo derecho (`self.rect.right`) se encuentra

fuera de la pantalla.

```
# TODO (2.6): Destruir a los enemigos
if self.rect.right < 0:
    self.kill()
```

Segunda parte de update() de Enemy en elements/enemy.py

Con esto tenemos lista la lógica del **Player** como la de **Enemy**. Pero si corremos el código...

No pasa NADA???

2.7 - Jorge y los BUGS Parte 2 (TODO)

Nuestros sprites ya tienen una lógica impecable, solo nos falta agregarlos a nuestra escena.

Nos vamos a **game_scene** y, en primer lugar, creamos la instancia de nuestro **Player**. Esto resulta fácil, ya que solo debe existir un único jugador.

```
# TODO (2.7): Crear la instancia de jugador
player = Player(screen)
```

scenes/game_scene.py

Una vez creado, similar a lo que hicimos con el fondo, tenemos que dibujarlo en nuestra pantalla. Nuevamente, como **Player** es único, esto resulta sencillo; podríamos simplemente llamar al método **.blit()** y ya. Pero como lo vamos a hacer cuando tengamos docenas o cientos de instancias de **Enemy**?

Tendremos que instanciar y llamar **.blit()** para cada uno individualmente?

Para evitar este dolor de cabeza, vamos a utilizar una funcionalidad de **pygame** llamada **Group**. Estos son una especie de lista especial de **pygame** que nos permiten agrupar sprites y actuar sobre ellos de forma simultanea. Primero, creamos los grupos con **pygame.sprite.Group()**


```
# TODO (2.7): Crear los grupos de sprites
enemies = pygame.sprite.Group()
all_sprites = pygame.sprite.Group()
all_sprites.add(player)
```

scenes/game_scene.py

Creamos un **Group** para los enemigos y uno para todos los sprites. Para agregar un sprite al grupo, llamamos **grupo.add(sprite)**, muy similar a **lista.append(elemento)**. Aprovechamos de agregar nuestra instancia de **Player** al grupo **all_sprites**.

Ahora que tenemos un **Group** que contiene a todos los sprites, resulta mucho mas fácil poder actualizar y dibujar a todos al mismo tiempo.

Para actualizarlos, utilizamos

```
# TODO (2.7): Actualizar el estado interno de los sprites (posiciones, etc)
pressed_keys = pygame.key.get_pressed()
player.update(pressed_keys)
enemies.update()
```

scenes/game_scene.py

No podemos llamar **all_sprites.update()** ya que **Player** necesita la lista de teclas presionadas, la cual obtenemos con **pygame.key.get_pressed()**. Con **enemies** no hay problema, así que llamamos **enemies.update()** directamente.

Una vez actualizados todos los sprites, tenemos que dibujarlos en la pantalla.

```
# TODO (2.7): Dibujar los sprites actualizados en la ventana
for entity in all_sprites:
    screen.blit(entity.image, entity.rect)
```

scenes/game_scene.py

Que hace **.blit()**? Toma dos argumentos, el QUE se dibujara y el DONDE se dibujara en la superficie que escojamos, en este caso, **screen**. Si recuerdan, a los enemigos les definimos su posición inicial directamente en su **rect**, por lo que usamos **entity.rect** para el DONDE. **Player** usara el **(0, 0)** por defecto.

Con esto, ya podrán mover a **Jorge**!

2.8 – Generando BUGS (TODO)

Habrán notado que al correr el juego, no han aparecido los **BUGS**. Esto se debe a que no hemos creado ninguna instancia de estos. A diferencia de **Player**, vamos a tener muchísimos BUGS, por lo que en lugar de instanciarlos individualmente, vamos a crear un evento que los genere.

pygame utiliza un sistema interno para manejar **eventos**, asignándoles un número entero hasta 32. En sí, la librería ya utiliza los primeros 23 eventos, por lo que tenemos disponibles desde el **24 al 32**. Para crear nuestro evento, tenemos que asignarle un valor libre, en ese caso usamos

pygame.USEREVENT (variable cuyo valor es 24) + 1. Si queremos crear más eventos, usaríamos **pygame.USEREVENT + 2**, **pygame.USEREVENT + 3**, etc...

Entonces, en nuestra **game_scene**

```
# TODO (2.8): Crear el generador de enemigos
ADDENEMY = pygame.USEREVENT + 1
pygame.time.set_timer(ADDENEMY, 600)
```

scenes/game_scene.py

Que hace **pygame.time.set_timer()**? Recibe el **evento que será emitido** y **cada cuantas milésimas de segundo emitirlo**. En este caso, se emite el evento **ADDENEMY** cada **600 ms**.

Y ahora que tenemos el evento creado, podemos comenzar a generar los **BUGS**.

```
# Iteramos sobre cada evento en la cola
for event in pygame.event.get():
    if event.type == KEYDOWN: # se presiono una tecla?
        if event.key == K_ESCAPE: # era la tecla de escape?
            running = False # terminamos el loop

    elif event.type == QUIT: # fue un click al cierre de la ventana?
        running = False # terminamos el loop

    # TODO (2.8): Generar enemigos
    elif event.type == ADDENEMY:
        new_enemy = Enemy(screen)
        enemies.add(new_enemy)
        all_sprites.add(new_enemy)
```

scenes/game_scene.py

Si corren **main.py**, ya deberían aparecer los enemigos! Pero...

2.9 - El Tiempo (TODO)

Todo pasa muy rápido, no?

El problema que tenemos es que el juego esta funcionando tan rápido como el procesador del computador lo permita. Necesitamos limitar esto, por lo que utilizaremos un reloj de juego con **pygame.time.Clock()**

```
# TODO (2.9): Crear el reloj del juego
clock = pygame.time.Clock()
```

scenes/game_scene.py

Luego, dentro del gameloop debemos indicarle al juego cada cuanto actualizarse. Como es estándar, definiremos 60 actualizaciones por segundo.

```
# TODO (2.9): Controlar la velocidad de fotogramas
clock.tick(60)
```

scenes/game_scene.py

Si corremos el juego ahora, todo ocurre a una velocidad razonable!

2.10 - Esquivando BUGS (TODO)

Solo nos queda un objetivo, si **Jorge** colisiona con algún **BUG**, el juego debe finalizar.

Para esto, pygame nuevamente vuelve al rescate con la función **pygame.sprite.spritecollideany()**, la cual automáticamente detecta colisiones entre un sprite y cualquier sprite que pertenece a un grupo dado.

```
# TODO (2.10): Calcular colisiones entre jugador y enemigos
if pygame.sprite.spritecollideany(player, enemies):
    player.kill()
    running = False
```

Con esto, nuestro juego esta listo!

3 - JUGAR

Ya has programado tu primer juego!

3.1 - Para el Entregable (TODO)

Ahora debes hacer 2 pequeños cambios al código, así nos demuestras que te diste el tiempo de leer el manual y no solo copiaste el código.

Para el entregable además del juego completo debes modificarlo:

- Hacer que Jorge se mueva con las teclas **WASD** en vez de las flechas
- Hacer que los bugs se muevan **verticalmente** (aparezcan abajo de la pantalla y se muevan hacia arriba)

Lo que se explica en este manual es suficiente para que puedas hacer esos cambios. No necesitas investigar en internet.

Aun tienes tiempo? Trata de mejorarlo y añadirle funciones para personalizarlo!!! (podría ser útil para una experiencia futura)